Building a Highly Available, Scalable Web Application Project

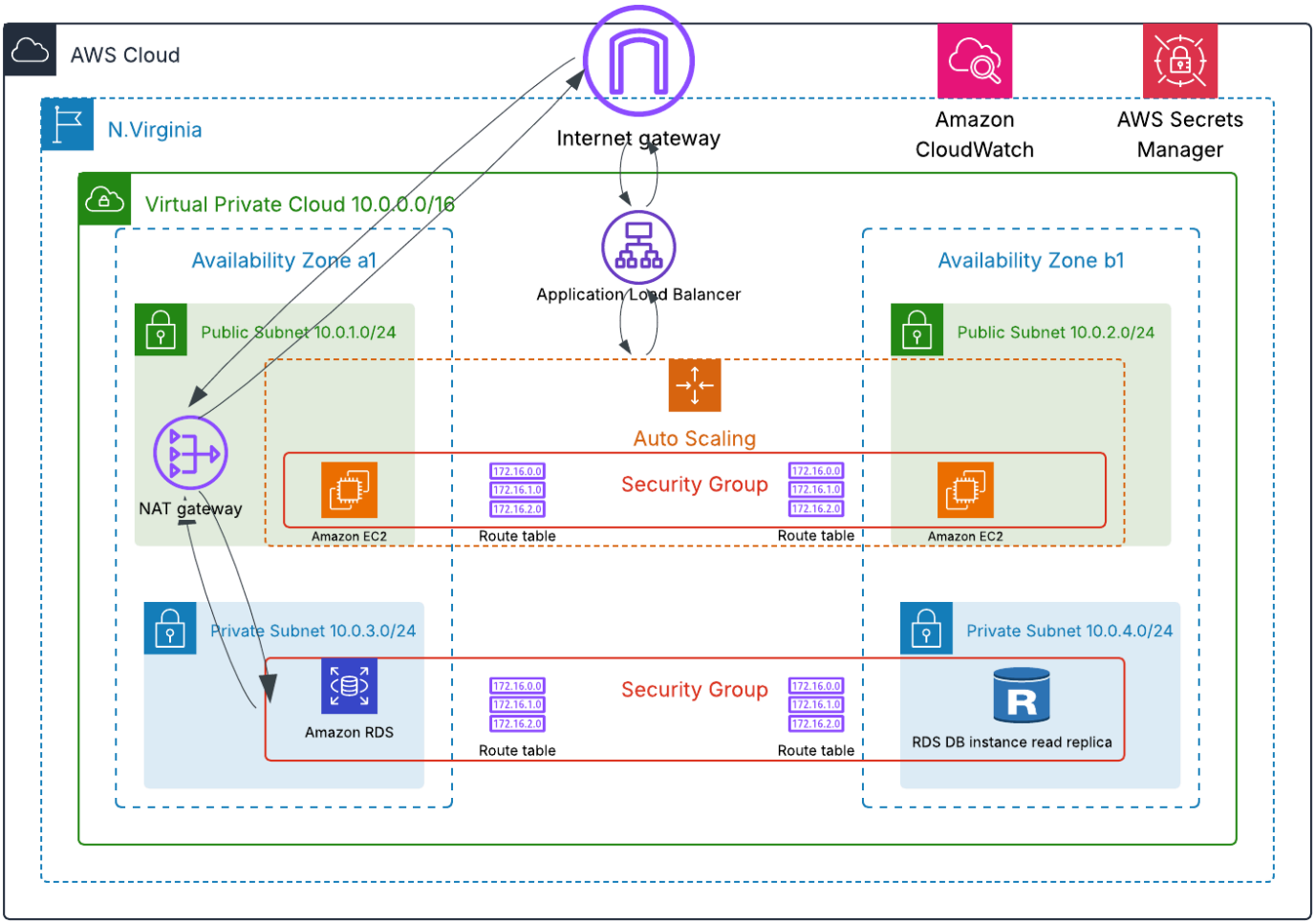
- Name: Menna Allah Ahmed Ibarhim Eldosky
- Group: CAI1_SWD8_M2d
- Email: mennaahmed0640@gmail.com
- Cell Phone: 01092593895 or 01151901449 ●

LinkedIn:www.linkedin.com/in/menna-ahmed-620151219

Requirements:

- Create an architectural diagram to depict various AWS services and their interactions with each other.
- Estimate the cost of using services by using the AWS Pricing Calculator.
- Deploy a functional web application that runs on a single virtual machine and is backed by a relational database.
- Architect a web application to separate layers of the application, such as the web server and database.
- Create a virtual network that is configured appropriately to host a web application that is publicly accessible and secure. ● Deploy a web application with the load distributed across multiple web servers.
- Configure the appropriate network security settings for the web servers and database.
- Implement high availability and scalability in the deployed solution.
- Configure access permissions between AWS services.

1- Architecture Diagram and Cost Estimation:



2-Estimate Costs:

A cost projection for operating the architecture on AWS for a 12-month period is generated using the AWS Pricing Calculator to ensure accurate budgeting.

Contact your AWS representative: Contact Sales

Export date: **10/17/2024** Language: **English**

Estimate URL: **https://calculator.aws/#/estimate?id=a860323bb70440c33698dde8a5afe64f4bac574f**

Estimate summary

Upfront cost	Monthly cost	Total 12 months cost
196.22 USD	1,163.11 USD	14,153.54 USD Includes upfront cost

Detailed Estimate

Name	Group	Region	Upfront cost	Monthly cost
Amazon EC2	No group applied	US East (N. Virginia)	196.22 USD	85.05 USD
Status: - **Description:** **Config summary:** Tenancy (Shared Instances), Operating system (Linux), Workload (Consistent, Number of instances: 4), Advance EC2 instance (t3a.small), Pricing strategy (1yr Partial Upfront), Enable monitoring (enabled), EBS Storage amount (40 GB), DT Inbound: Internet (50 GB per month), DT Outbound: Internet (50 GB per month), DT Intra-Region: (0 TB per month)				
Elastic Load Balancing	No group applied	US East (N. Virginia)	0.00 USD	40.85 USD
Status: - **Description:** **Config summary:** Number of Application Load Balancers (2)				

Abstract:

The purpose of this project is to design, build, and deploy a highly available and scalable web application using Amazon Web Services (AWS). The project involves implementing a multi-tier architecture, where the web application and database are decoupled to ensure better performance and flexibility. In the initial phase, the project creates a virtual private cloud (VPC) with public and private subnets across two Availability Zones, ensuring redundancy and fault tolerance. The web application is hosted on an Amazon EC2 instance, while the database is provisioned separately using Amazon RDS to manage and store data efficiently.

To improve the security and efficiency of communication between the application and database, AWS Secrets Manager is used to manage database credentials securely. AWS Cloud9 provides a development environment for managing application code, running AWS CLI commands, and migrating databases. Auto Scaling and Load Balancer services are configured to ensure the application scales seamlessly based on traffic, offering both high availability and cost optimization.

This project demonstrates the use of cloud-native services for building a scalable, secure, and robust web application architecture, suitable for real-world deployment scenarios.

I. Introduction:

1. Overview of the project

This project focuses on developing a highly available, scalable, and secure web application hosted on the Amazon Web Services (AWS) cloud platform. The application architecture follows a 3-tier design with web, application, and database layers. The project is built using AWS services like EC2 for hosting the web application, RDS for managing the database, and Auto Scaling for handling traffic spikes.

2. Objectives of the solution

The primary objectives of this project include:

- **High Availability:** Ensure the web application is accessible even in case of failure or maintenance in one availability zone by distributing the resources across multiple zones.

- **Scalability:** Automatically scale the application based on traffic demand, ensuring the performance remains optimal during peak loads.

- **Security:** Protect the web application and database using AWS security features like security groups, private subnets, and IAM roles.
- **Cost-effectiveness:** Utilize AWS services efficiently to minimize costs without sacrificing performance or availability.
- Automation: Use Infrastructure as Code (IaC) principles for automating the deployment and scaling of the application.

3. Functional Requirements

- **Web Application Hosting:** The application should be hosted on an EC2 instance, which serves dynamic content to users via a web interface.

- **Database Management:** The backend database must be hosted separately on Amazon RDS (MySQL engine) to decouple the application components and improve performance.

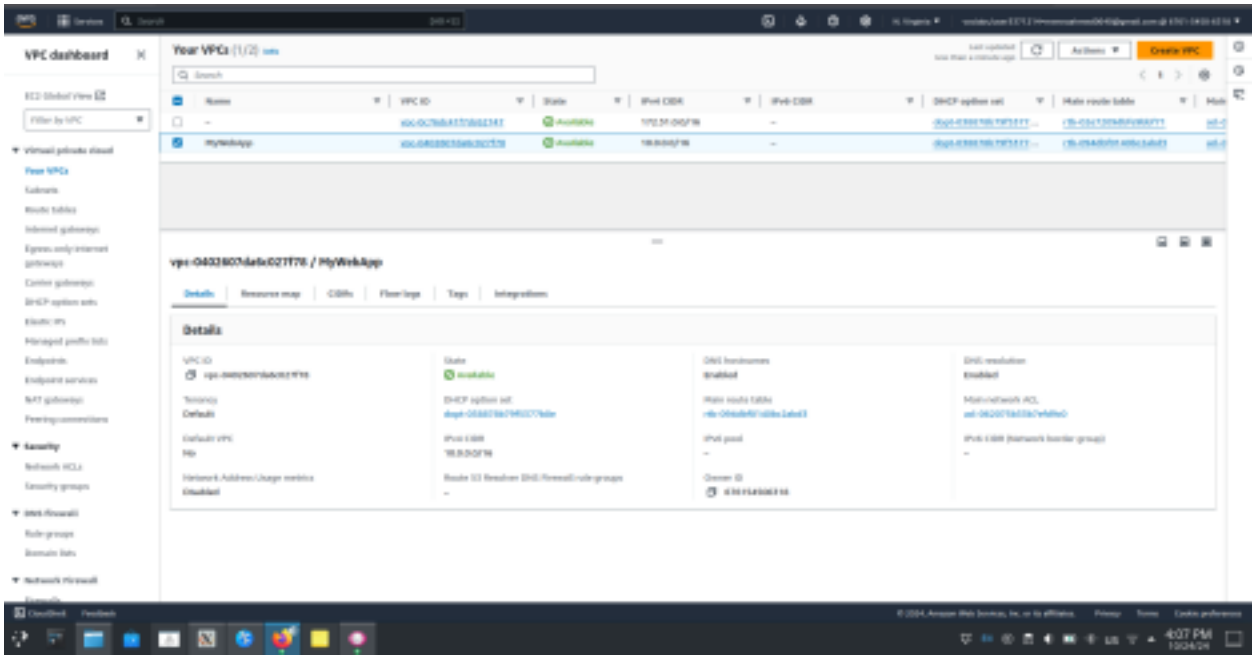
- **Scalability:** The system must support auto-scaling to handle increased loads dynamically and reduce resources during low usage.

- **Security Controls:** Implement security controls to restrict access to the database from the public internet, allowing only the web application to interact with it.

- **Monitoring and Logging:** Utilize AWS CloudWatch for monitoring application performance and health, and set up logging for tracking access and performance metrics.

2-Implementation Steps:

A.VPC

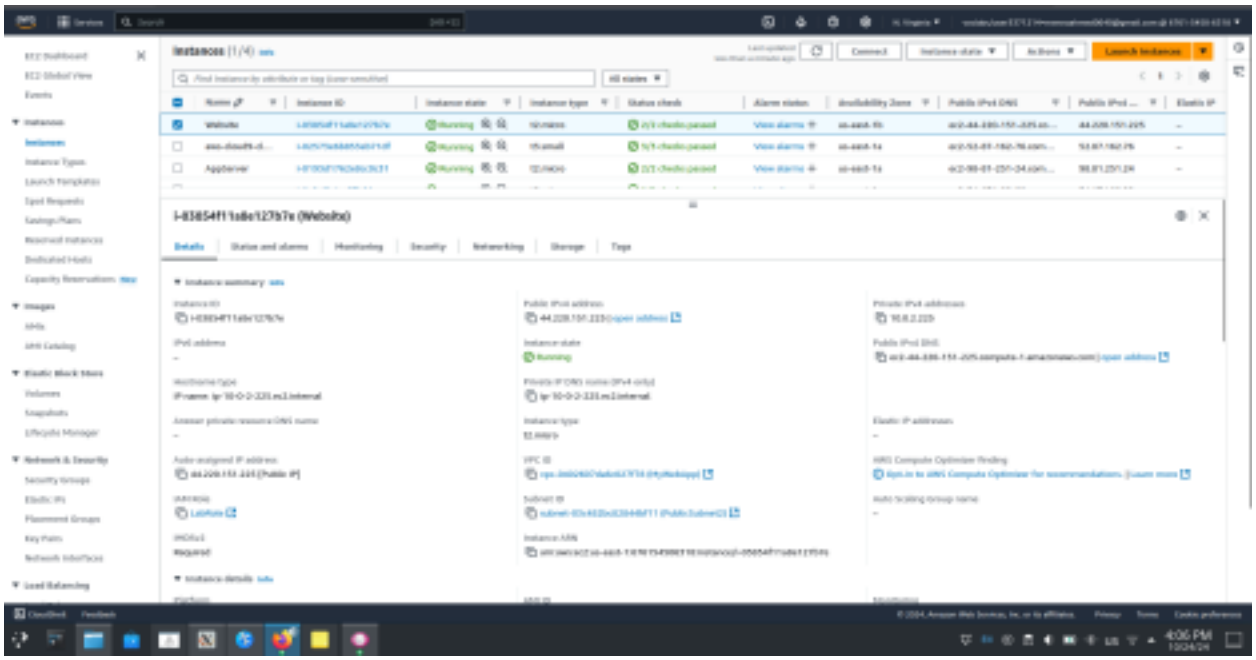


This screenshot shows the VPC (Virtual Private Cloud) dashboard in AWS. The selected VPC is named "MyWebApp," and its VPC ID is displayed. Key details include:

- **VPC CIDR block**: 10.0.0.0/16, defining the IP address range for the network.
- **State:** Available, indicating the VPC is active.
- **DNS resolution**: Enabled, allowing resources within the VPC to resolve domain names.
- **Main route table**: Displays the ID of the route table managing traffic within the VPC.
- **DHCP option set:** A set of rules to assign network configuration to instances in the VPC.

This VPC is a custom network where your web application and database will reside, ensuring isolation and security within AWS.

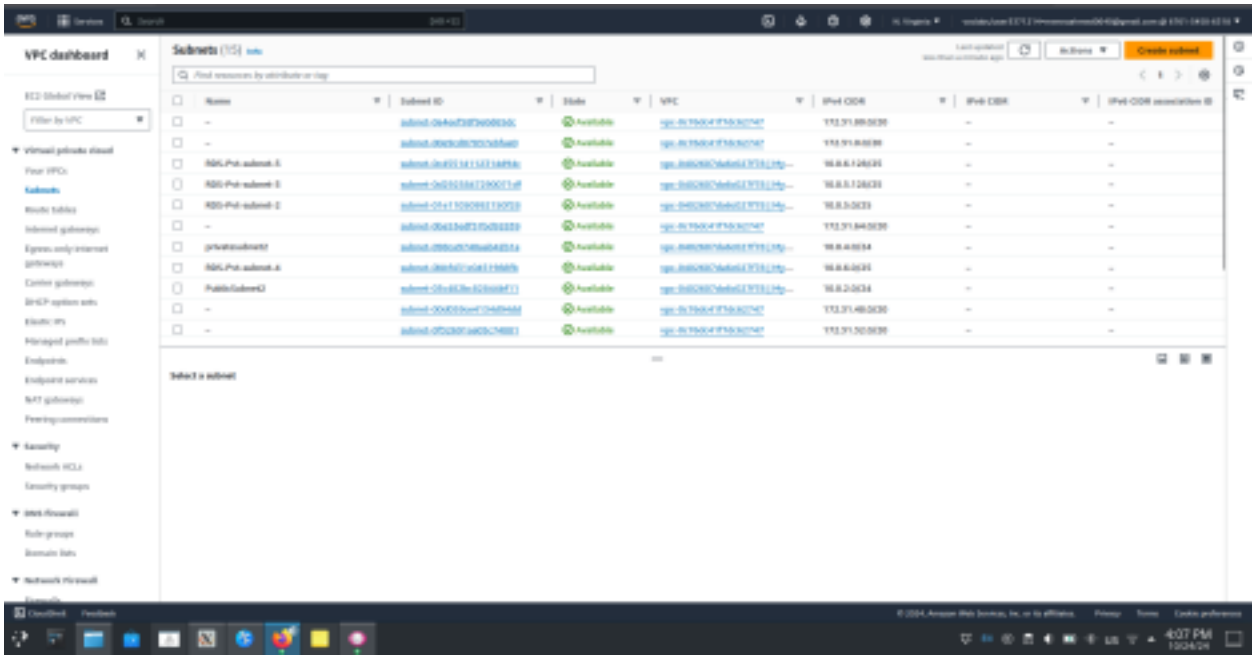
B. **EC2 Instance**



This screenshot shows details of an EC2 instance named "Website" running in AWS. Key information includes:

- **Instance ID:** I-03854f11a8e127b7e
- **Instance state:** Running (indicating it is currently active) ● **Instance type**: t2.micro (a small instance type used for low-cost general-purpose workloads)
- **Private IP**: 10.0.2.225 (used for internal communication within the VPC)
- **Public IP:** 44.220.151.225 (used to access the instance from the internet)
- **VPC ID**: vpc-0402607da6c027f78 (shows the instance belongs to the "MyWebApp" VPC)
- **Subnet ID**: subnet-03c482bc8284bff11 (indicates which subnet the instance is deployed in)
- I**AM Role:** LabRole (this role provides permissions for the instance to interact with AWS services)

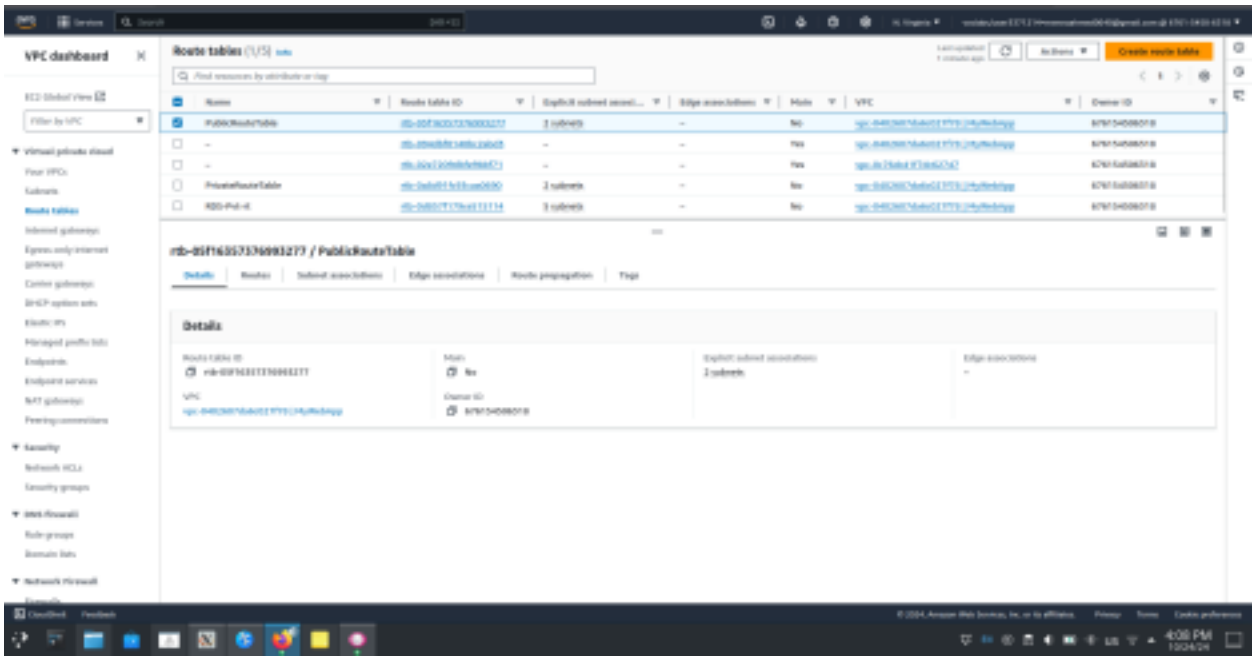
C. **Subnets**



This screenshot shows the list of subnets within your Virtual Private Cloud (VPC) in AWS. Key points to note:

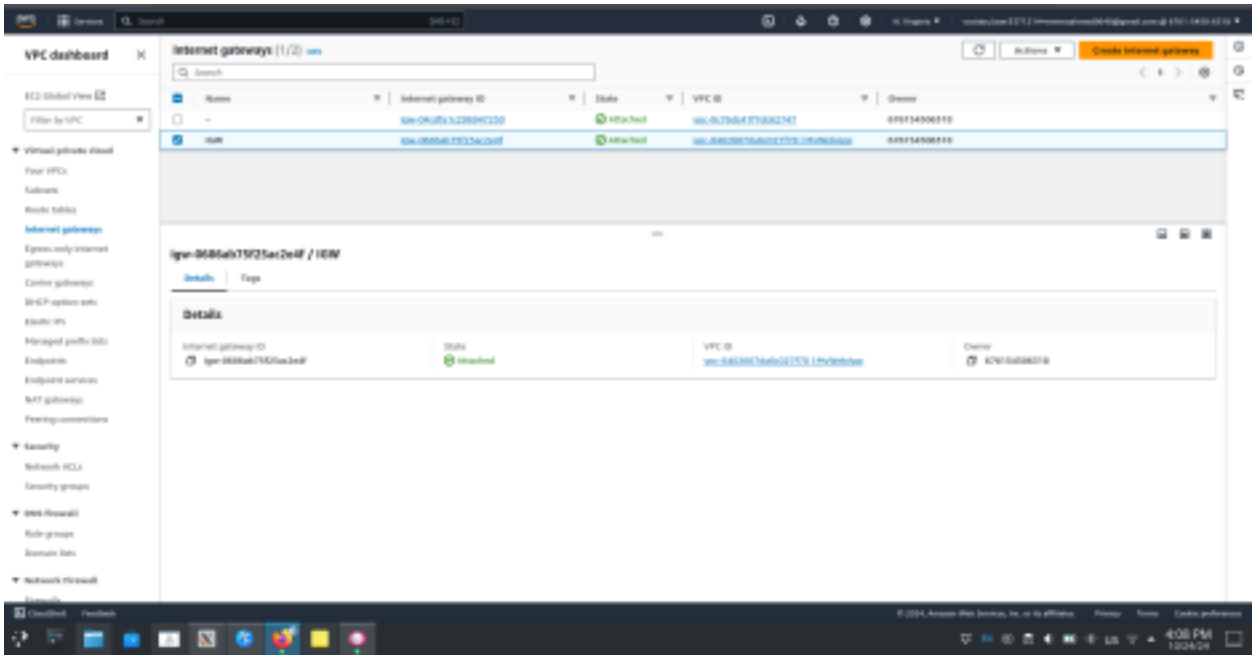
- **Subnet names**: You have subnets like `RDS-Pvt-subnet-3`, `PublicSubnet2`, and `privatesubnet2`, indicating both public and private subnets.
- **Subnet IDs**: Unique identifiers for each subnet (e.g., `subnet-04aef58f9e0d65dc` for the first subnet).
- **State**: All subnets are in the "Available" state, meaning they are ready for use.
- **VPCs**: Most subnets are associated with either your "MyWebApp" VPC (`vpc-0402607da6c027f78`) or another VPC (`vpc-0c76dc41f7dc62747`).
- **IPv4 CIDR blocks**: The IP range assigned to each subnet (e.g., `10.0.0.0/24`, `172.31.80.0/20`). Public subnets typically allow internet traffic, while private subnets are used for internal resources, like databases.

D. **Route Tables**



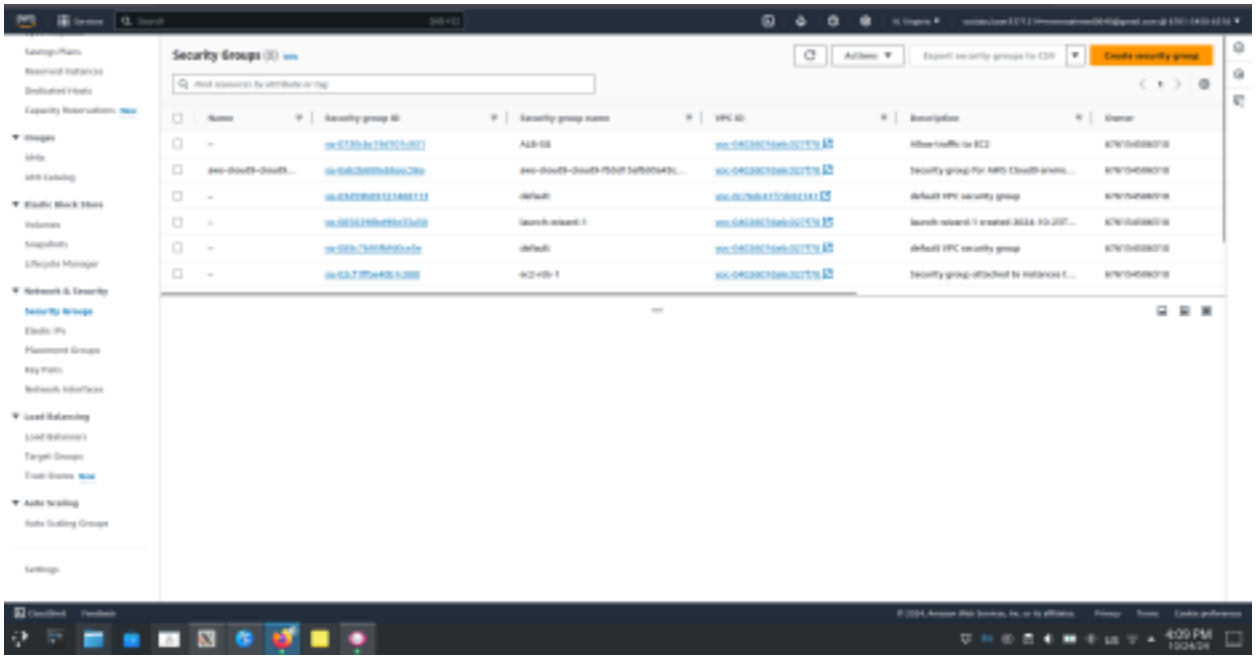
- **Route Tables:**These are network objects that control how traffic flows within a VPC. The list shows the route table ID, name, number of explicit subnet associations, and other relevant information.
- **Public Route Table:**This is a special type of route table that allows traffic to flow between the VPC and the public internet.
- **Details**:Clicking on a route table likely provides more detailed information about its configuration, including routes, subnet associations, and propagation settings.
- **Actions:**The "Create route table" button suggests that you can create new route tables within this VPC.

E. Internet Gateways



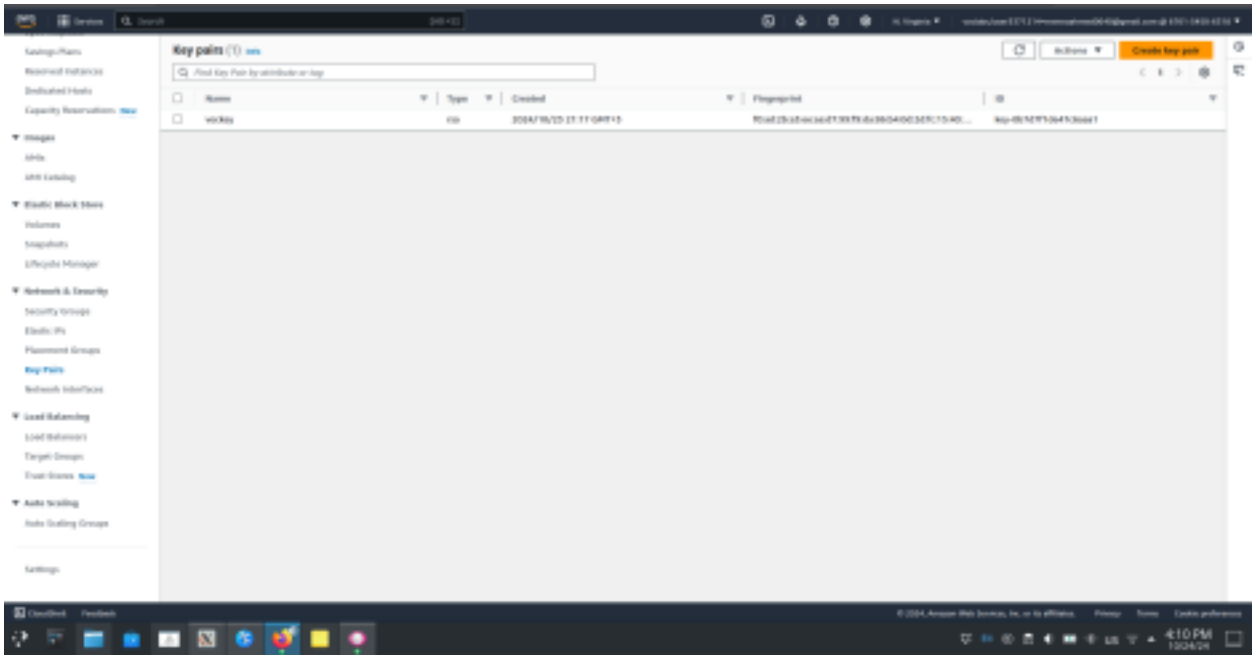
- **Internet Gateways:**These are network objects that allow traffic to flow between your VPC and the public internet. The list shows the internet gateway ID, state, VPC ID, and owner information.
- **Details:**Clicking on an internet gateway likely provides more detailed information about its configuration, including the internet gateway ID, state, VPC ID, and owner.
- **Actions:**The "Create internet gateway" button suggests that you can create new internet gateways within this VPC.

F. Security Groups



- **Security Groups:** These are network objects that act as virtual firewalls, controlling inbound and outbound traffic for resources within a VPC. The list shows the security group ID, name, VPC ID, and description.
- **Action:** The "Create security group" button suggests that you can create new security groups within this VPC.

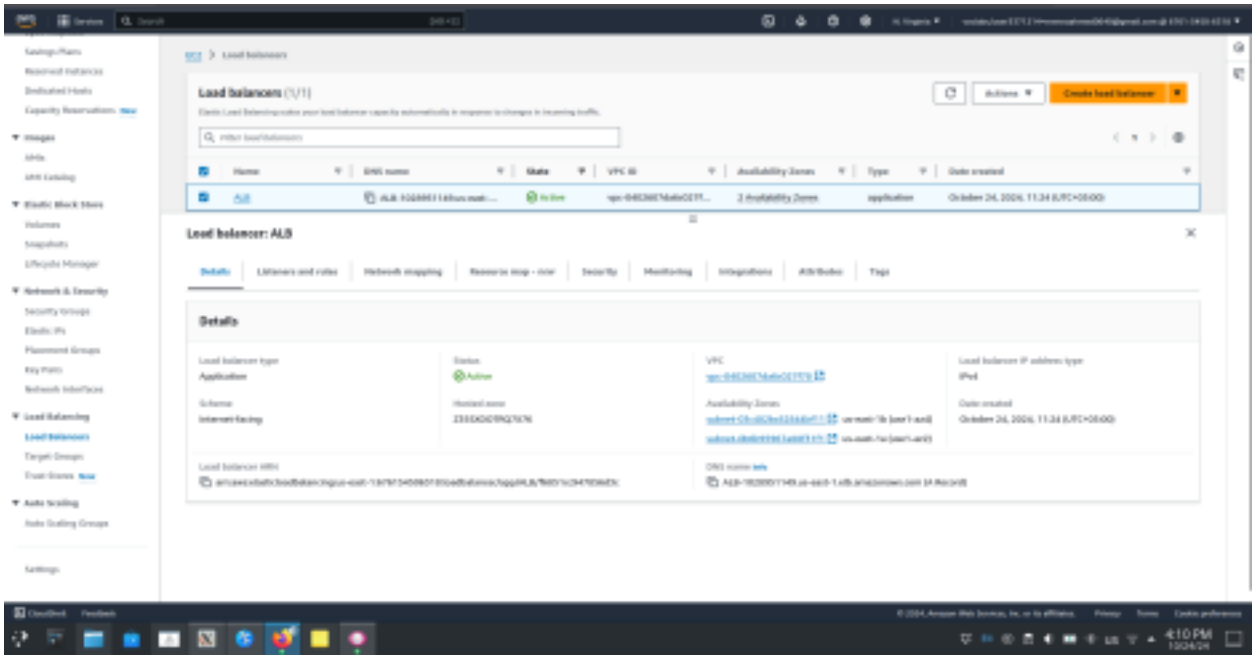
G. Key Pairs



- **Key Pairs:** These are cryptographic key pairs used for authentication and encryption. The list shows the key pair name, type, created date, and fingerprint.
- **Actions:** The "Create key pair" button suggests that you can create new key pairs within this VPC.

Overall, this image provides a basic overview of the key pairs present in the VPC and allows you to manage authentication and encryption for resources within your AWS environment.

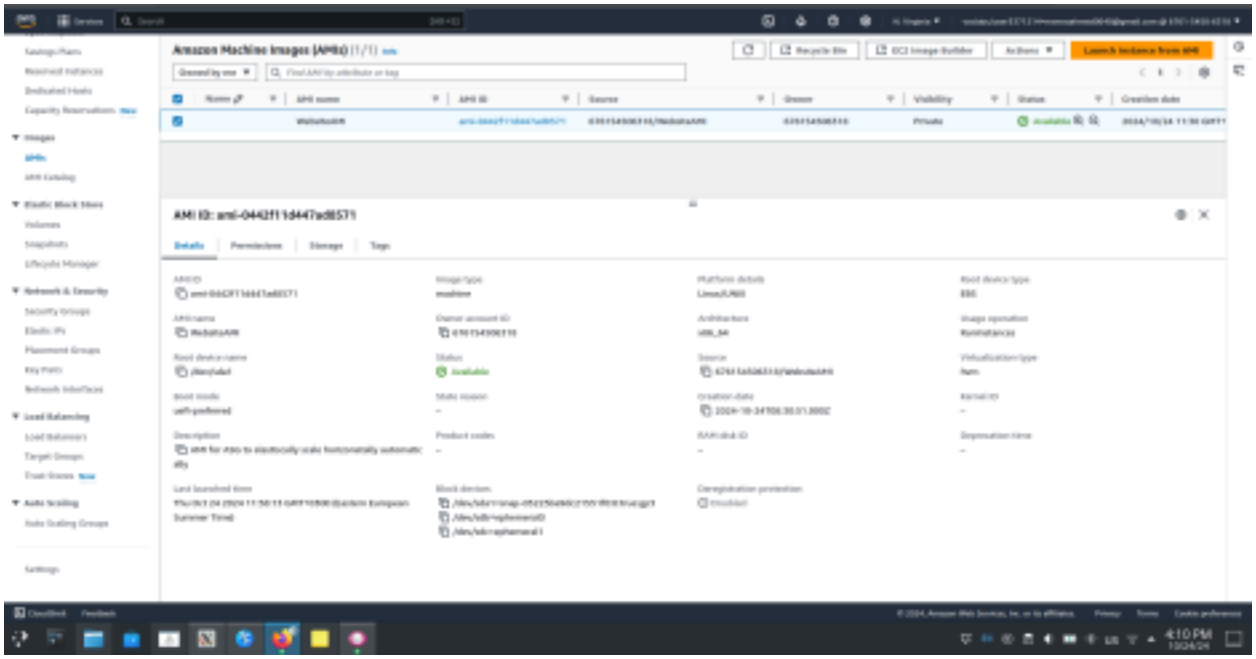
H. Load Balancers



- **Load Balancers:** These are network objects that distribute incoming traffic across multiple EC2 instances. The list shows the load balancer name, DNS name, state, VPC ID, and other relevant information.
- **Details:** Clicking on a load balancer likely provides more detailed information about its configuration, including listeners, rules, security settings, and health checks.
- **Actions:** The "Create load balancer" button suggests that you can create new load balancers within this VPC.

Overall, this image provides a basic overview of the load balancers present in the VPC and allows you to manage traffic distribution across your EC2 instances.

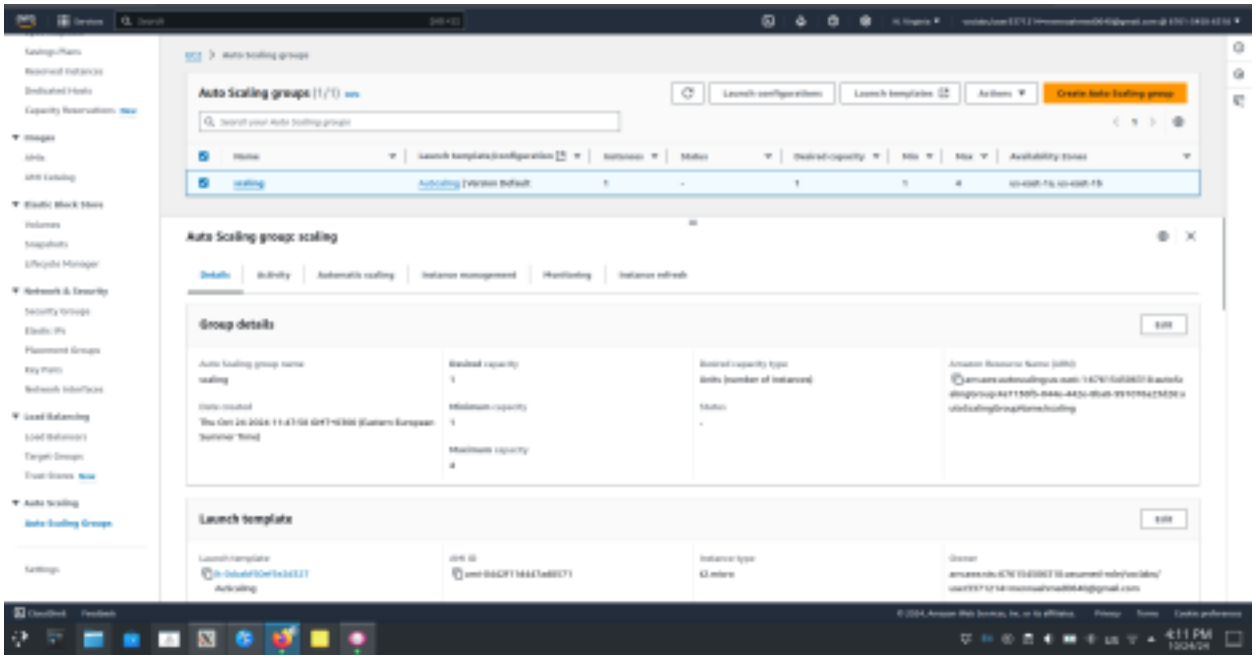
I. AMIs



- **AMIs:** These are templates that define the software configuration for an EC2 instance. The list shows the AMI ID, name, owner, and other relevant information.
- **Details:** Clicking on an AMI likely provides more detailed information about its configuration, including platform details, root device type, and kernel ID.
- **Actions:** The "Launch instance from AMI" button suggests that you can create new EC2 instances based on this AMI.

Overall, this image provides a basic overview of the AMIs available in the VPC and allows you to launch new EC2 instances with specific configurations.

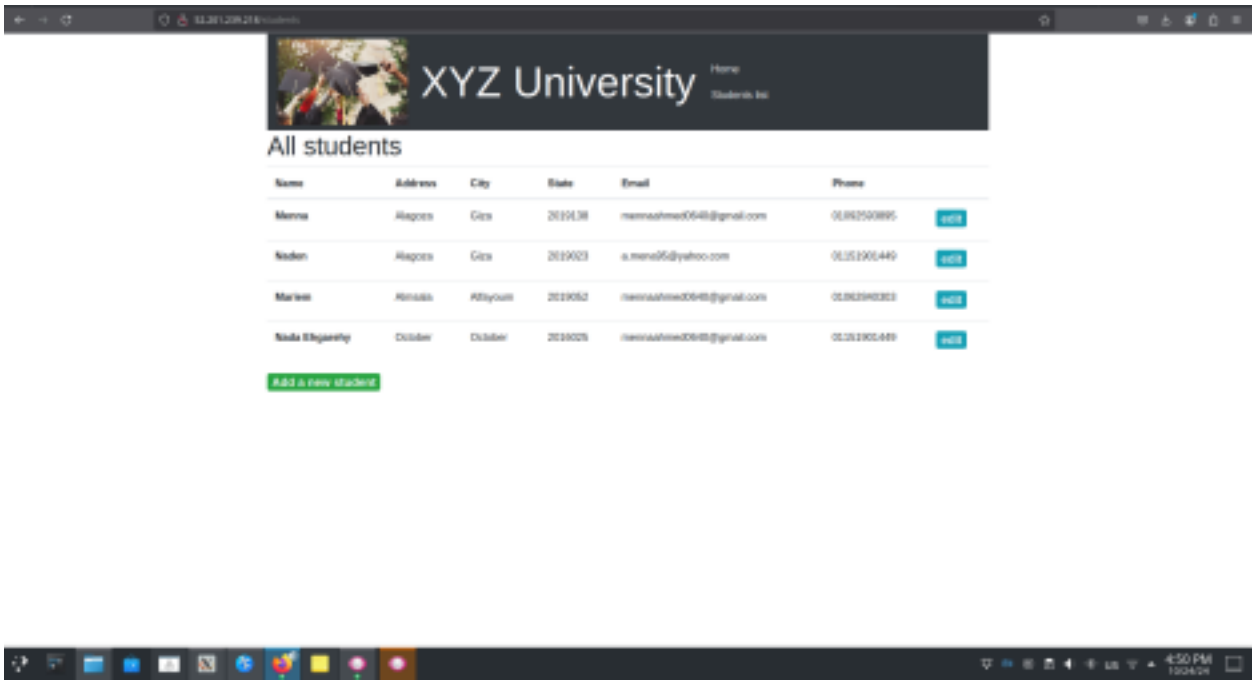
J. Auto Scaling Groups



- **Auto Scaling Groups:** These are collections of EC2 instances that automatically scale up or down based on demand. The list shows the Auto Scaling group name, launch template, instances, and other relevant information.
- **Details:** Clicking on an Auto Scaling group likely provides more detailed information about its configuration, including desired capacity, minimum and maximum capacity, and health check settings.
- **Actions:** The "Create Auto Scaling group" button suggests that you can create new Auto Scaling groups within this VPC.

Overall, this image provides a basic overview of the Auto Scaling groups present in the VPC and allows you to manage the scaling of your EC2 instances based on demand.

K. Final Testing Deployment:



In conclusion, this project successfully demonstrates the design and implementation of a highly available, scalable web application using AWS cloud infrastructure. By leveraging multiple AWS services such as VPC, EC2, RDS, Auto Scaling, and Cloud9, the application is designed to handle traffic fluctuations while ensuring performance and resilience. The architecture follows best practices for security, cost-effectiveness, and scalability, making it an ideal solution for production-level deployment. Additionally, AWS Secrets Manager ensures secure handling of sensitive data, enhancing the overall security of the system. This project highlights the power of cloud technology in building modern, efficient web applications.